

Experiment - 5

Create SQL views for department salary summary and employee hierarchy. Test updatability of views. Implement a recursive CTE to display reporting chains.

1. Database and Schema Setup (DDL & DML)

-- Step 1: Create and select the database

```
CREATE DATABASE IF NOT EXISTS LabPracticalDB; USE
LabPracticalDB;
```

-- Step 2: Create Department Table

```
CREATE TABLE Department ( dept_id
INT PRIMARY KEY, dept_name
VARCHAR(50) NOT NULL
);
```

-- Step 3: Create Employee Table (with manager_id for hierarchy)

```
CREATE TABLE Employee ( emp_id INT
PRIMARY KEY, emp_name
VARCHAR(100) NOT NULL,
salary DECIMAL(10, 2),
dept_id INT,
manager_id INT,
FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)
);
```

-- Step 4: Insert Departments

```
INSERT INTO Department (dept_id, dept_name) VALUES
(101, 'Engineering'),
(102, 'Data Science'),
(103, 'Human Resources');
```

-- Step 5: Insert Employees (Notice the manager_id assignments establishing a chain of command)

-- Employee 1 is the CEO (Manager is NULL)

```
INSERT INTO Employee (emp_id, emp_name, salary, dept_id, manager_id) VALUES
(1, 'Aarav Sharma', 150000, 101, NULL),
(2, 'Vihaan Singh', 120000, 101, 1),
(3, 'Ananya Gupta', 115000, 102, 1),
(4, 'Advik Desai', 85000, 101, 2),
(5, 'Myra Reddy', 95000, 102, 3),
(6, 'Diya Kumar', 75000, 103, 1),
(7, 'Ishaan Mehta', 65000, 101, 4),
(8, 'Kabir Das', 60000, 102, 5);
```

2. Creating SQL Views

Views are virtual tables based on the result-set of an SQL statement.

Query 1: Department Salary Summary View Creates a summarized view of financial metrics per department.

```
CREATE VIEW Dept_Salary_Summary AS
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS total_employees,
    SUM(e.salary) AS total_salary_expense,
    ROUND(AVG(e.salary), 2) AS average_salary
FROM Department d
LEFT JOIN Employee e ON d.dept_id = e.dept_id GROUP
BY d.dept_name;
```

-- Execute this to see the view's output:

```
SELECT * FROM Dept_Salary_Summary;
```

The screenshot shows the ByteXL SQL editor interface. The left pane displays the SQL code for creating a database schema and a view. The right pane shows the output of the view query.

SQL Code:

```
6 CREATE TABLE Department (
7 );
8
9 -- Step 3: Create Employee Table (with manager_id for hierarchy)
10 CREATE TABLE Employee (
11     emp_id INT PRIMARY KEY,
12     emp_name VARCHAR(100) NOT NULL,
13     salary DECIMAL(10, 2),
14     dept_id INT,
15     manager_id INT,
16     FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
17     FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)
18 );
19
20 -- Step 4: Insert Departments
21 INSERT INTO Department (dept_id, dept_name) VALUES
22 (101, 'Engineering'),
23 (102, 'Data Science'),
24 (103, 'Human Resources');
25
26 -- Step 5: Insert Employees (Notice the manager_id assignments establishing a chain of command)
27 -- Employee 1 is the CEO (Manager is NULL)
28 INSERT INTO Employee (emp_id, emp_name, salary, dept_id, manager_id) VALUES
29 (1, 'Aarav Sharma', 150000, 101, NULL),
30 (2, 'Vihaan Singh', 120000, 101, 1),
31 (3, 'Ananya Gupta', 115000, 102, 1),
32 (4, 'Advik Desai', 85000, 101, 2),
33 (5, 'Myra Reddy', 95000, 102, 3),
34 (6, 'Diya Kumar', 75000, 103, 1),
35 (7, 'Ishaan Mehta', 65000, 101, 4),
36 (8, 'Kabir Das', 60000, 102, 5);
37
38 CREATE VIEW Dept_Salary_Summary AS
39 SELECT
40     d.dept_name,
41     COUNT(e.emp_id) AS total_employees,
42     SUM(e.salary) AS total_salary_expense,
43     ROUND(AVG(e.salary), 2) AS average_salary
44 FROM Department d
45 LEFT JOIN Employee e ON d.dept_id = e.dept_id
46 GROUP BY d.dept_name;
47
48 -- Execute this to see the view's output:
49 SELECT * FROM Dept_Salary_Summary;
```

Output:

dept_name	total_employees	total_salary_expense	average_salary
Engineering	4	420000.00	105000.00
Data Science	3	270000.00	90000.00
Human Resources	1	75000.00	75000.00

Query 2: Employee Hierarchy View Creates a view using a self-join to easily see who reports to whom.

```
CREATE VIEW Employee_Hierarchy AS
SELECT
    e.emp_id AS Employee_ID,
    e.emp_name AS Employee_Name,
    m.emp_name AS Manager_Name
FROM Employee e
LEFT JOIN Employee m ON e.manager_id = m.emp_id;
-- Execute this to see the view's output:
SELECT * FROM Employee_Hierarchy;
```

The screenshot shows the ByteXL Editor interface with a SQL editor on the left and an output panel on the right. The SQL editor contains two queries. The first query, labeled 'Step 5: Insert Employees', inserts data into an 'Employee' table and creates a view 'Dept_Salary_Summary'. The second query creates a view 'Employee_Hierarchy'. The output panel shows the results of these queries, including a table of department statistics and a hierarchy table.

```

25 (102, 'Data Science');
26 (103, 'Human Resources');
27
28 -- Step 5: Insert Employees (Notice the manager_id assignments establishing a chain of command)
29 -- Employee 1 is the CEO (Manager is NULL)
30 INSERT INTO Employee (emp_id, emp_name, salary, dept_id, manager_id) VALUES
31 (1, 'Aarav Sharma', 150000, 101, NULL),
32 (2, 'Vihaan Singh', 120000, 101, 1),
33 (3, 'Ananya Gupta', 115000, 102, 1),
34 (4, 'Advik Desai', 85000, 101, 2),
35 (5, 'Myra Reddy', 95000, 102, 3),
36 (6, 'Diya Kumar', 75000, 103, 1),
37 (7, 'Ishaan Mehta', 65000, 101, 4),
38 (8, 'Kabir Das', 60000, 102, 5);
39
40 CREATE VIEW Dept_Salary_Summary AS
41 SELECT
42     d.dept_name,
43     COUNT(e.emp_id) AS total_employees,
44     SUM(e.salary) AS total_salary_expense,
45     ROUND(AVG(e.salary), 2) AS average_salary
46 FROM Department d
47 LEFT JOIN Employee e ON d.dept_id = e.dept_id
48 GROUP BY d.dept_name;
49
50 -- Execute this to see the view's output:
51 SELECT * FROM Dept_Salary_Summary;
52
53 CREATE VIEW Employee_Hierarchy AS
54 SELECT
55     e.emp_id AS Employee_ID,
56     e.emp_name AS Employee_Name,
57     m.emp_name AS Manager_Name
58 FROM Employee e
59 LEFT JOIN Employee m ON e.manager_id = m.emp_id;
60
61 -- Execute this to see the view's output:
62 SELECT * FROM Employee_Hierarchy;

```

Output:

dept_name	total_employees	total_salary_expense	average_salary
Engineering	4	420000.00	105000.00
Data Science	3	270000.00	90000.00
Human Resources	1	75000.00	75000.00

Employee_ID	Employee_Name	Manager_Name
1	Aarav Sharma	NULL
2	Vihaan Singh	Aarav Sharma
3	Ananya Gupta	Aarav Sharma
4	Advik Desai	Vihaan Singh
5	Myra Reddy	Ananya Gupta
6	Diya Kumar	Aarav Sharma
7	Ishaan Mehta	Advik Desai
8	Kabir Das	Myra Reddy

3. Testing Updatability of Views

Test 1: Attempting to update a Complex View (This will produce an ERROR)

-- This fails because the database cannot map a changed average back to a specific row.

```

UPDATE Dept_Salary_Summary
SET average_salary = 100000
WHERE dept_name = 'Engineering';

```

Test 2: Creating and updating a Simple View (This will SUCCEED)

-- Create a simple view with a direct row-to-row mapping

```

CREATE VIEW Engineering_Staff AS
SELECT emp_id, emp_name, salary
FROM Employee
WHERE dept_id = 101;

```

-- This update succeeds because there are no aggregations or group by clauses

```

UPDATE Engineering_Staff
SET salary = salary + 5000
WHERE emp_id = 7;

```

-- Check the base table to confirm the data actually changed

```

SELECT emp_name, salary FROM Employee WHERE emp_id = 7;

```

4. Recursive CTE for Reporting Chains

This query starts at the top (CEO) and recursively joins the table to itself to build a visual text chain of command for every employee.

WITH RECURSIVE ReportingChain AS (

```

-- Anchor Member: Start with the top-level manager (manager_id is NULL)
SELECT
    emp_id,
emp_name,
manager_id,
    1 AS hierarchy_level,
    CAST(emp_name AS CHAR(255)) AS chain_of_command
FROM Employee
WHERE manager_id IS NULL

UNION ALL

-- Recursive Member: Join the Employee table with the CTE itself
SELECT
    e.emp_id,
    e.emp_name,
    e.manager_id,
rc.hierarchy_level + 1,
    CONCAT(rc.chain_of_command, ' -> ', e.emp_name)
FROM Employee e
INNER JOIN ReportingChain rc ON e.manager_id = rc.emp_id
)
-- Display the generated paths
SELECT
    emp_name,
    hierarchy_level,
    chain_of_command
FROM ReportingChain
ORDER BY hierarchy_level, emp_id;

```

The screenshot shows the byteXL SQL editor interface. The left pane contains the SQL query, and the right pane displays the results of the query execution.

SQL Query:

```

72 -- This update succeeds because there are no rows with emp_id = 7
73 UPDATE Engineering_Staff
74 SET salary = salary + 5000
75 WHERE emp_id = 7;
76
77 -- Check the base table to confirm the update
78 SELECT emp_name, salary FROM Employee WHERE emp_id = 7;
79
80 WITH RECURSIVE ReportingChain AS (
81     -- Anchor Member: Start with the top-level manager (manager_id is NULL)
82     SELECT
83         emp_id,
84         emp_name,
85         manager_id,
86         1 AS hierarchy_level,
87         CAST(emp_name AS CHAR(255)) AS chain_of_command
88     FROM Employee
89     WHERE manager_id IS NULL
90
91     UNION ALL
92
93     -- Recursive Member: Join the Employee table with the CTE itself
94     SELECT
95         e.emp_id,
96         e.emp_name,
97         e.manager_id,
98         rc.hierarchy_level + 1,
99         CONCAT(rc.chain_of_command, ' -> ', e.emp_name)
100    FROM Employee e
101    INNER JOIN ReportingChain rc ON e.manager_id = rc.emp_id
102 )
103 -- Display the generated paths
104 SELECT
105     emp_name,
106     hierarchy_level,
107     chain_of_command
108 FROM ReportingChain
109 ORDER BY hierarchy_level, emp_id;

```

Query Results:

emp_name	salary
Ishaan Mehta	70000.00

emp_name	hierarchy_level	chain_of_command
Aarav Sharma	1	Aarav Sharma
Vihaan Singh	2	Aarav Sharma -> Vihaan Singh
Ananya Gupta	2	Aarav Sharma -> Ananya Gupta
Diya Kumar	2	Aarav Sharma -> Diya Kumar
Advik Desai	3	Aarav Sharma -> Vihaan Singh -> Advik Desai
Myra Reddy	3	Aarav Sharma -> Ananya Gupta -> Myra Reddy
Ishaan Mehta	4	Aarav Sharma -> Vihaan Singh -> Advik Desai -> Ishaan Mehta
Kabir Das	4	Aarav Sharma -> Ananya Gupta -> Myra Reddy -> Kabir Das